

2. SOFTWARE REQUIREMENTS SPECIFICATION

2.1 Types of Requirements

Software requirements are divided into two main types (i) Functional Requirements (ii) Non Functional Requirements. Functional requirements describe what the system should do, and Non-functional requirements describe how the system should behave in terms of performance, security, and reliability.

User Classes & Roles

Role	Access Level	Capabilities
Admin	Full	Supervise user accounts, perform CRUD on all entities, and modify configurations.
Data Entry Operator	Write + Read	Create or modify profiles for wildlife, including tracking, medical, and genetic data.
Researcher	Read + Export	Execute queries via portal/API and download records in CSV or JSON formats.
Government Official	Read Only	Limited to viewing and searching existing animal information and profiles.
Developer	API Read	Retrieve data from public endpoints using authenticated JWT tokens.

Assumptions & Dependencies

- Constant network connectivity is assumed for all users interacting with the web portal or API.
- WIS acts strictly as a storage repository for GPS data generated by external tracking hardware.
- The initial release (v1.0) will support the English language exclusively for all operations.

2.2 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 (or equivalent), 2.0 GHz
RAM	4 GB (8 GB recommended for development)
Storage	50 GB free disk space for database
Network	Stable internet connection for API access
Server (deployment)	2 CPU cores, 4 GB RAM, 100 GB SSD (cloud instance)

2.3 Software Requirements

Software	Purpose
Windows 10 / Ubuntu 20.04	Operating System
Node.js v18+	Backend runtime
PostgreSQL 14+	Database management
Postman	API testing
VS Code	Code editor
Git & Github	Version control
Browser (Chrome/Firefox)	Frontend testing

2.4 Functional Requirements

These functional requirements describes what the system must be able to do:

FR-01: Animal Profile Management

Users need to be able to put in new wildlife records and also change or remove them when something comes up. Details like species age and weight should get captured in there too. The system creates a unique ID by itself for each animal. I think checking that the entered data is valid has to happen as well. It feels like those parts connect but maybe not in a super strict order.

FR-02: Movement Data Recording

This function stores GPS coordinates and timestamps tied to animal IDs. It seems batch uploads from a CSV file are how multiple entries get added at once. The coordinates get checked before anything saves to confirm they are valid. That part feels important but some details on the verification step are not totally clear right away. Maybe the checks happen in a specific order that is easy to miss.

FR-03: DNA & Biological Data

The database is going to keep a record of all the DNA samples. This will include the results from the lab and the information that confirms what species it is. The DNA samples are connected to the animals profile. This is done using a computer program that matches the samples to the individual wildlife profile. The database will have all the information, about the DNA samples and the wildlife profiles. It will keep them linked together.

FR-04: Health & Veterinary Records

Users can keep track of their pets veterinary history. This includes: Vaccination records, Diagnosed conditions, Treatments given When keeping these records

users must fill in: The date of the service, A brief summary of what happened. This helps users keep their pets medical history organized. They can see what vaccinations their pets have had what conditions they have been diagnosed with and what treatments they have received. The veterinary history is easy to understand.

FR-05: RESTful API Access

Users can keep track of their pets medical history. This includes: Vaccination records, Diagnosed conditions, Treatments given, When adding a record users must fill in: Date of service, A brief summary, The system has a way for other programs to access its data. This is done through: Endpoints for resources, like animal movements and species, Data is sent in a JSON format , A special token is required for every request to ensure authorized access. The token is used for authentication.

FR-06: User Management

Administrators can control user accounts. They can give users jobs and take away access if needed. Passwords are kept safe using bcrypt hashing. Each user has to verify their email address, which's unique, to them. The system is designed to keep everything.

FR-07: Search & Filter

The system has a good search feature that lets users find what they need by looking at the type of species, where something is located or when it happened. The system shows the search results one page at a time which makes it easier for users to look at the species information location details or timeframes they searched for.

FR-08: CSV Bulk Upload

Data Entry Operators can perform bulk movement data imports. The system validates every row in the file, committing valid data and generating a report for any skipped entries.

FR-09: Data Export

Researchers can get their search results in formats like JSON or CSV. They can do this in two ways. One way is through the website. The other way is through API endpoints. This helps them to easily use their search results. They can choose how they want to work with their results. The export option is available on the web and, through API. So researchers can export results in JSON or CSV. It is easy to use for them. They have choices to get results in formats.

2.5 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Performance	API responses will be returned within 2 seconds for standard queries.
NFR-02	Scalability	The database shall handle up to 1 million animal records without performance loss.
NFR-03	Security	All API calls require JWT authentication. Passwords shall be hashed using bcrypt and also will use Salt method for better.
NFR-04	Availability	The system will be available 99% of the time after deployment.
NFR-05	Data Integrity	All records should have timestamps and audit logs showing who created or modified them.
NFR-06	Usability	There will be a simple web interface for non technical data entry operator to use without training.
NFR-07	Compatibility	The API will follow REST standards and work with any programming language or tool.

2.6 Use Cases

Use Case Diagram

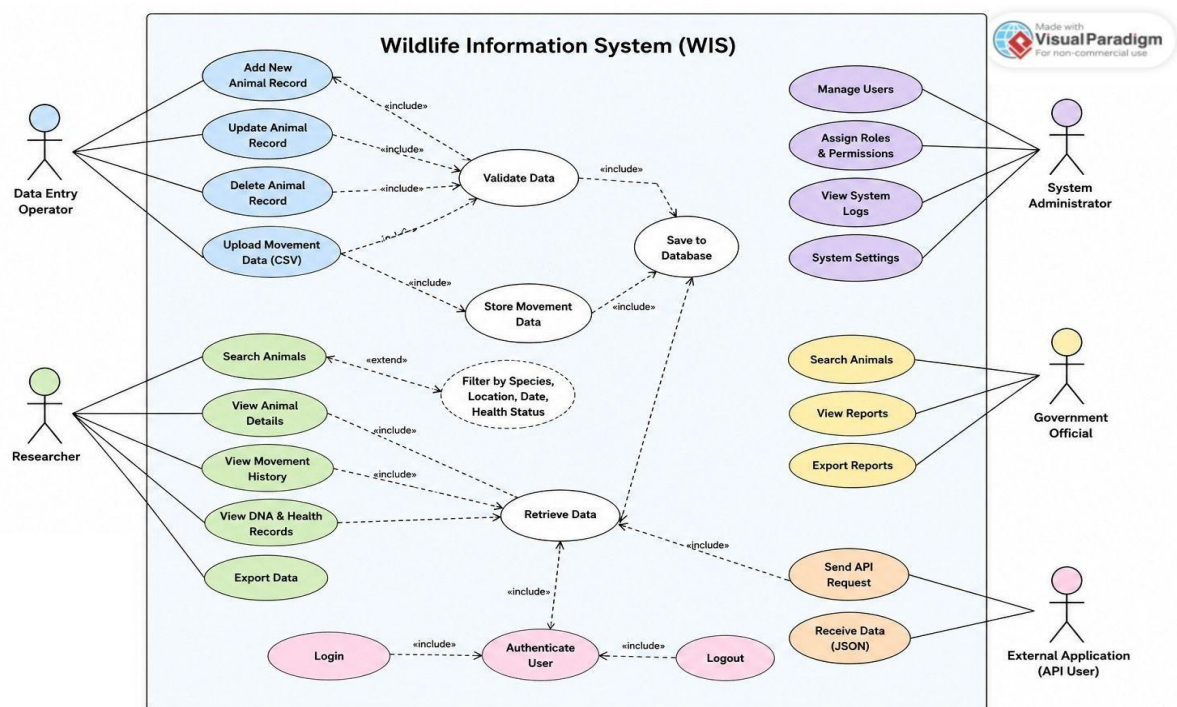


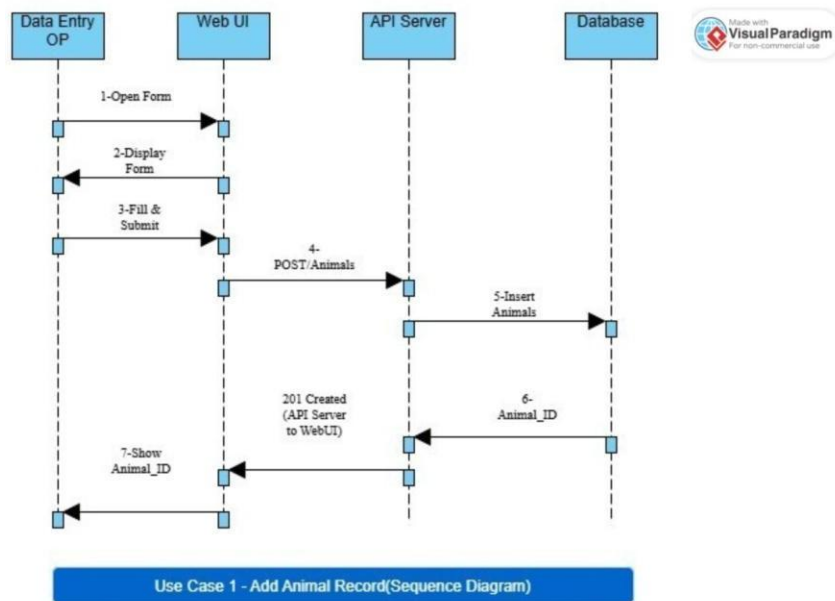
Figure 1: Use Case Diagram — Wildlife Information System

.1 Use Case Summary

UC ID	Name	Primary Actor	Category
UC-01	Add Animal Record	Data Entry Operator	Data Management
UC-02	Query Animal Data via API	Researcher / External App	API Access
UC-03	Upload CSV Movement Data	Data Entry Operator	Bulk Upload
UC-04	Search Animals	Researcher / Gov. Official	Search & Filter
UC-05	Manage Users	Admin	User Management
UC-06	View Health Records	Researcher / Vet	Data Access
UC-07	Export Data	Researcher	Data Export
UC-08	User Login / Authentication	All Users	Authentication
UC-09	Add DNA Sample Record	Data Entry Operator	Data Management

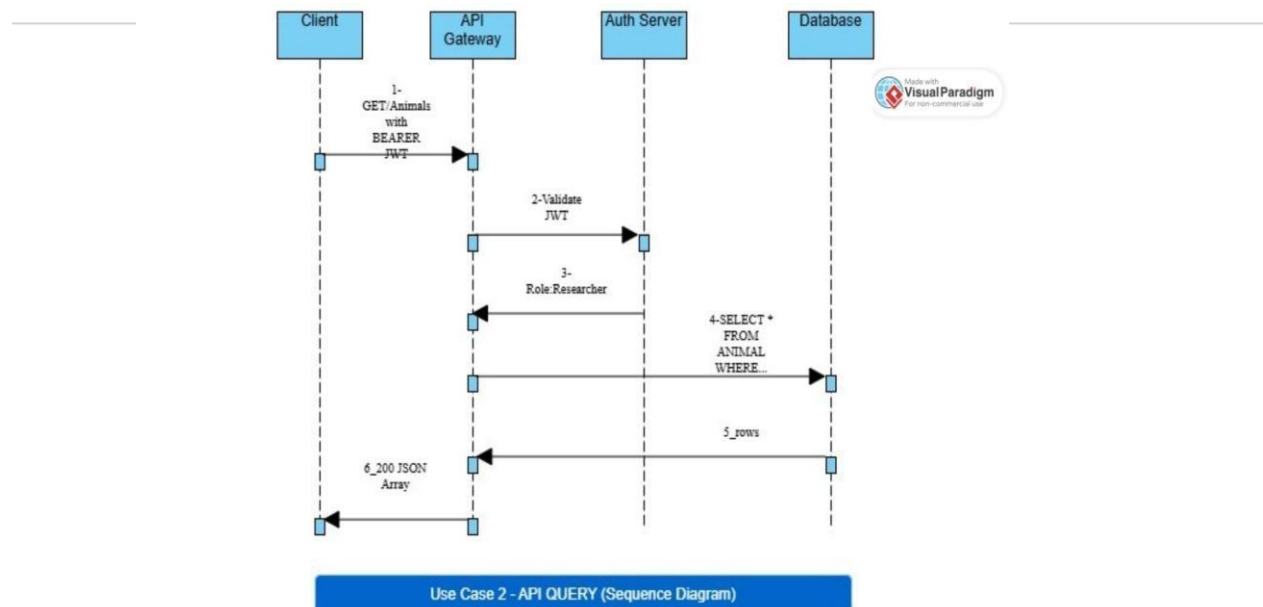
UC-01: Add Animal Record

Use Case ID	UC-01
Name	Add Animal Record
Actor	Data Entry Operator
Pre-condition	User is logged in with Data Entry or Admin role.
Post-condition	New animal record saved in the Animals table with a unique ID.
Main Flow	1. User opens Add Animal form. 2. System displays required fields. 3. User fills in species, name, gender, age, weight, location and submits. 4. System validates all fields. 5. System saves record. 6. System returns success message with new Animal ID.
Alternate Flow	If required fields are missing, system returns HTTP 400 with field level error messages. User corrects and resubmits.

Diagram: Add Animal Record => Sequence Diagram**UC-02: Query Animal Data via API**

Use Case ID	UC-02
Name	Query Animal Data via API
Actor	Researcher / External Application
Pre-condition	Client has a valid JWT token.
Post-condition	System returns JSON array of matching animal records with HTTP 200.
Main Flow	1. Client sends GET /api/animals?species=Snow+Leopard with Bearer token. 2. API Gateway validates JWT. 3. Auth Service confirms token and returns user role. 4. Query Controller passes parameters to Animal Service. 5. Animal Service executes SQL query. 6. Results formatted as JSON. 7. HTTP 200 returned with payload.
Alternate Flow	Invalid/expired token: HTTP 401 Unauthorized. Missing required query params: HTTP 400 Bad Request. No results found: HTTP 200 with empty array [].

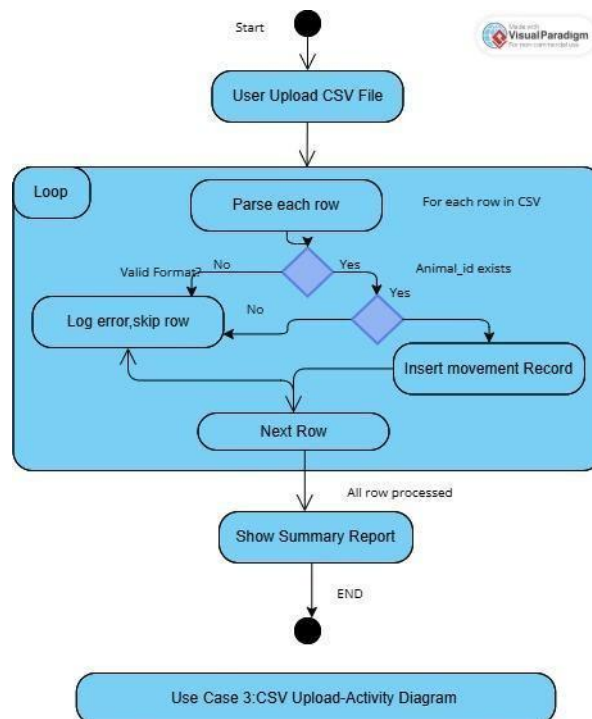
Diagram: API Query => Sequence Diagram



UC-03: Upload CSV Movement Data

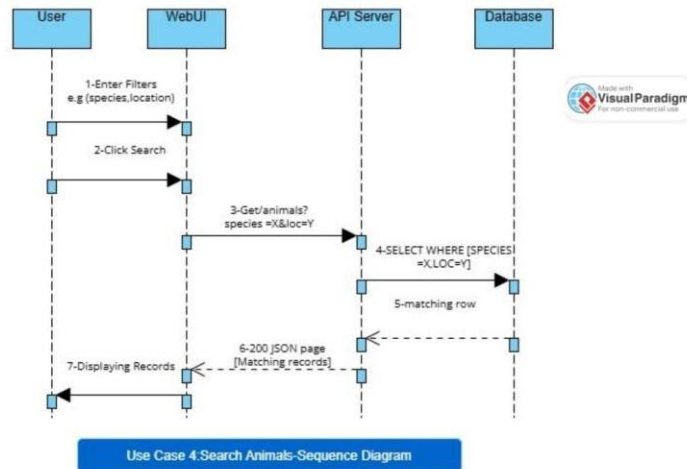
Use Case ID	UC-03
Name	Upload CSV Movement Data
Actor	Data Entry Operator
Pre-condition	User logged in. CSV file correctly formatted (animal_id, lat, lng, timestamp).
Post-condition	Valid rows saved as Movement Records linked to animal IDs. Upload report shown.
Main Flow	1. User navigates to Upload Movement Data page. 2. User selects and uploads CSV file. 3. System reads file and validates each row. 4. Valid rows inserted into Movement_Records table. 5. Summary report displayed: total rows, saved count, skipped rows with reasons.
Alternate Flow	Rows with missing/invalid coordinates are skipped. Rows with non-existent animal IDs are flagged. User sees per-row error report.

Diagram: CSV Upload => Activity Diagram



UC-04: Search Animals by Species or Location

Use Case ID	UC-04
Name	Search Animals by Species or Location
Actor	Researcher / Government Official
Pre-condition	User is logged in (any role).
Post-condition	Filtered, paginated list of matching animal records displayed.
Main Flow	1. User enters species name or location in search box. 2. User optionally sets date range filter. 3. User clicks Search. 4. System builds query with filters. 5. Database returns matching rows. 6. System displays paginated results (20 per page).
Alternate Flow	No records found: system displays 'No records found' message. If date range is invalid (end before start), system shows validation error.

Diagram: Search Animals => Sequence Diagram**UC-05: Manage Users (Admin)**

Use Case ID	UC-05
Name	Manage Users (Admin)
Actor	System Administrator
Pre-condition	User is logged in as Admin.
Post-condition	New user created or existing user role/status updated in Users table.
Main Flow	1. Admin opens User Management panel. 2. Admin clicks Create User or selects existing user. 3. Admin fills in name, email, role. 4. System validates email uniqueness. 5. System saves user; password hash generated. 6. Confirmation message shown.
Alternate Flow	Duplicate email: system returns error 'Email already exists'. Admin corrects and resubmits.

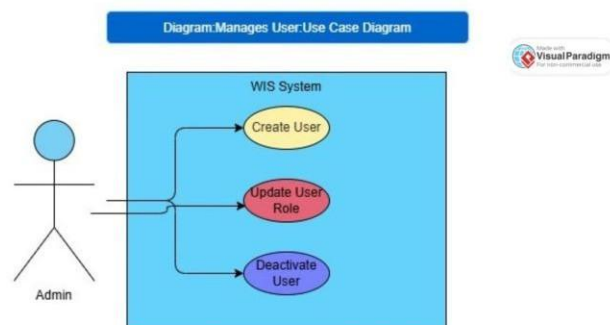
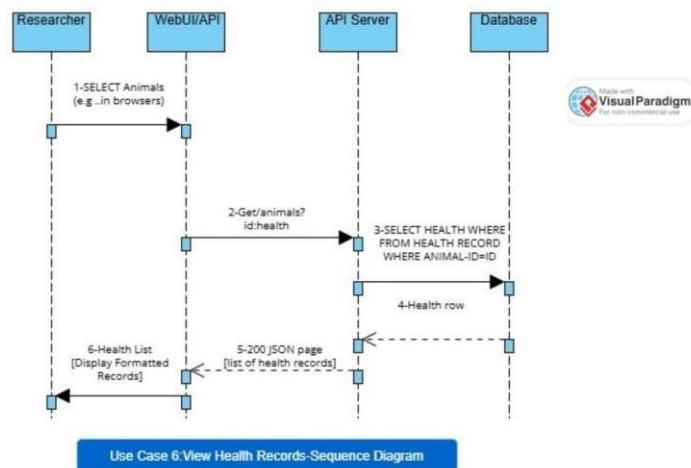


Diagram: Manage Users (Admin) => Use Case Diagram

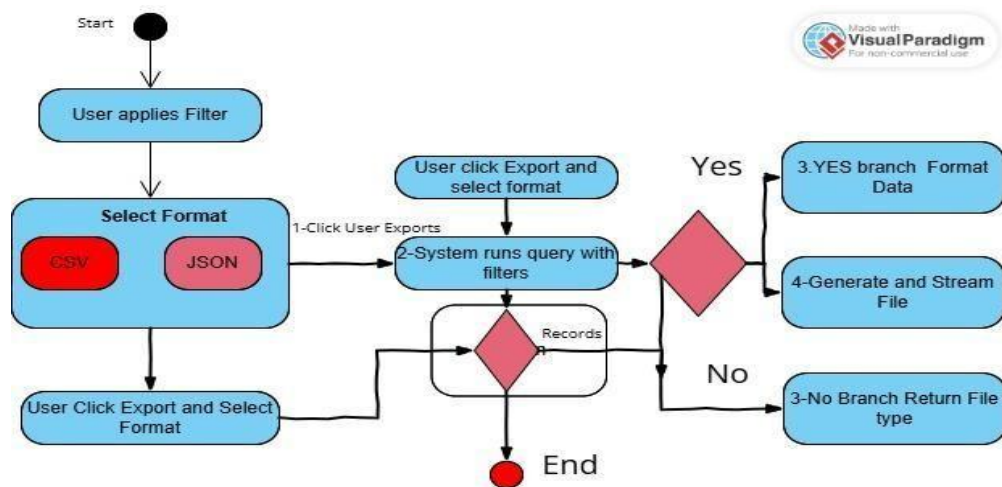
UC-06: View Health Records

Use Case ID	UC-06
Name	View Health Records
Actor	Researcher / Veterinarian
Pre-condition	User logged in with Researcher or Admin role. Animal ID known.
Post-condition	Health record list for the selected animal returned.
Main Flow	1. User opens Animal Detail page or sends GET /api/animals/:id/health. 2. System retrieves all health records for that animal ID. 3. Records sorted by date descending. 4. System returns list with condition, treatment, vet name, vaccination status.
Alternate Flow	Animal ID not found: HTTP 404. No health records exist: returns empty array with HTTP 200.

Diagram: View Health Records => Sequence Diagram

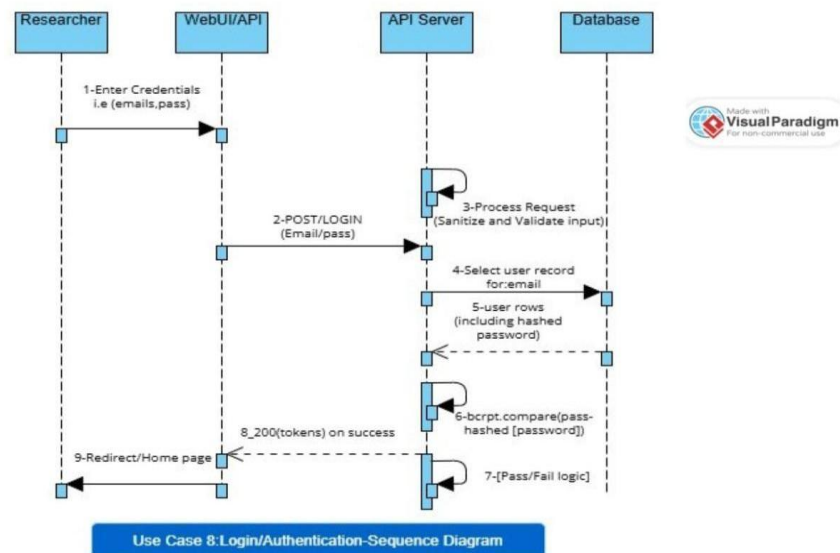
UC-07: Export Data

Use Case ID	UC-07
Name	Export Data
Actor	Researcher
Pre-condition	User logged in. Search/filter applied (optional).
Post-condition	CSV or JSON file downloaded by the user.
Main Flow	1. User performs search (optional). 2. User clicks Export and selects format (CSV or JSON). 3. System executes the query with current filters. 4. System formats data and generates file. 5. File downloaded to user's device.
Alternate Flow	No records match filters: system exports empty file with headers only and notifies user.

Diagram: Export Data => Activity Diagram**Diagram:Export Data:Activity Diagram**

UC-08: User Login / Authentication

Use Case ID	UC-08
Name	User Login / Authentication
Actor	All Users
Pre-condition	User has a registered account in the system.
Post-condition	User receives a valid JWT token used for subsequent API calls.
Main Flow	1. User opens login page or sends POST /api/auth/login with email and password. 2. System looks up user by email. 3. System compares password with bcrypt hash. 4. If match, system generates JWT with user role embedded. 5. Token returned with expiry time.
Alternate Flow	Wrong password: HTTP 401 Unauthorized. Non-existent email: HTTP 401 (generic, no user enumeration). Account deactivated: HTTP 403 Forbidden.

Diagram: Login / Authentication => Sequence Diagram

UC-09: Add DNA Sample Record

Use Case ID	UC-09
Name	Add DNA Sample Record
Actor	Data Entry Operator
Pre-condition	User logged in with Data Entry or Admin role. Animal ID exists in the system.
Post-condition	DNA sample record saved and linked to the specified animal.
Main Flow	1. User opens DNA Records section and selects an animal. 2. User fills in: sample ID, collection date, lab name, result, species confirmed flag. 3. User submits form. 4. System validates required fields and verifies animal ID. 5. System saves DNA record. 6. Confirmation with sample ID shown.
Alternate Flow	Animal ID not found: HTTP 404. Duplicate sample ID: HTTP 409 Conflict. Missing required fields: HTTP 400 with field errors.

Diagram: Add DNA Sample => Use Case Diagram